



College of Computing and Information Sciences

Name: Espartero, Ralph Isra P.

Section: III – CCSAD

Name: Arcilla, Andre Victor A.

Date: 04 - 15 - 2026

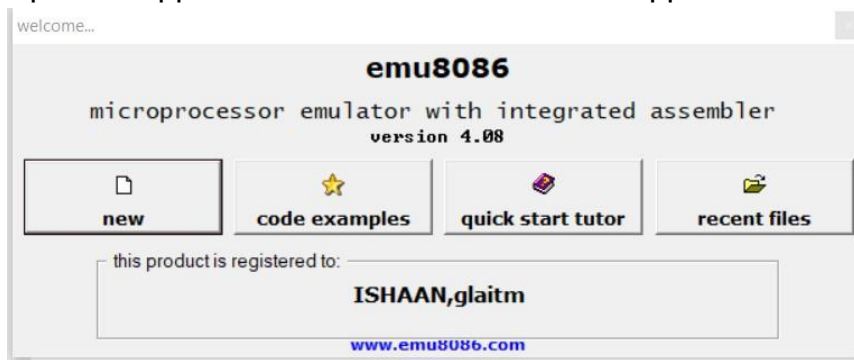
ARCHITECTURE AND ORGANIZATION

Activity 3: Intro to Assembly

1. Two members for Activity 3.
2. Install the emu8086/8088 on your desktop or laptop.
3. Once installed, in desktop, there shall be a logo that looks like the image below.



4. Open the application. A welcome window will appear. Choose New.



5. Another window will appear. This time, click the radio button COM template. This pertains to an executable file format which is a pure machine code.
6. The code window will appear. By default it contains the following code:

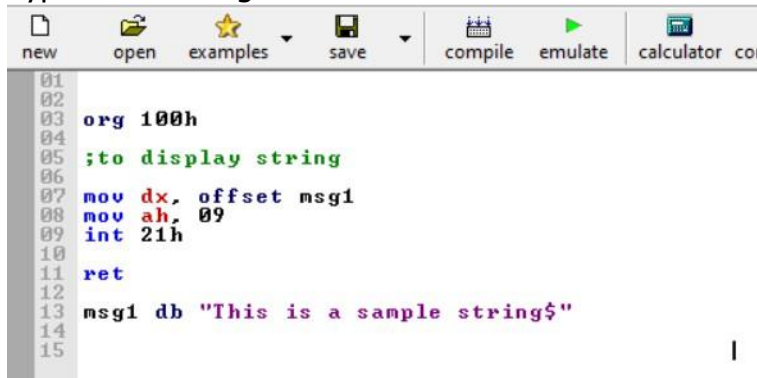
```
org 100h
;insert code here
ret
```

```
01
02 ; You may customize this and other start-up templates;
03 ; The location of this template is c:\emu8086\inc\0_com_template.txt
04
05 org 100h           ;use semicolon for comment
06                   ; org 100h means organize your code
07                   ; in memory allocation 100h <h means hexadecimal>
08                   ; all memory addresses uses hexadecimal numbering system
09
10
11
12
13 ret               ; ret means return control to the Operating System
14                   ; since we are directly accessing the main memory when coding in
15                   ; assembly, after running the program, we shall return control
16                   ; to the OS.
17
18
```

- Use semicolon for comment (;)
- **org 100h** means **organize your code in memory allocation 100h** (h means hexadecimal)

- All memory addresses use hexadecimal numbering system
- **ret** means **return control to the Operating System** since we are directly accessing the main memory when coding in assembly, after running the program, we shall return control to the OS.

7. Type the following in the code window:



```

01
02
03 org 100h
04
05 ;to display string
06
07 mov dx, offset msg1
08 mov ah, 09
09 int 21h
10
11 ret
12
13 msg1 db "This is a sample string$"
14
15

```

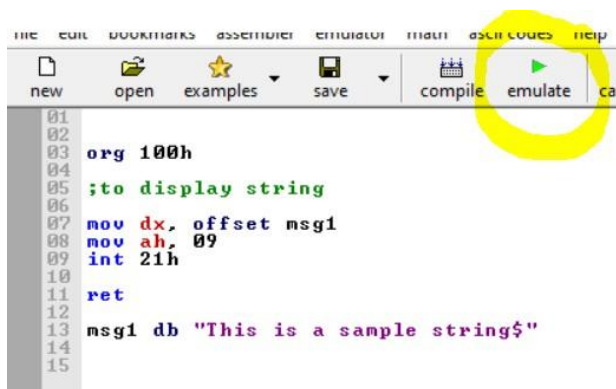
CPU has four general registers (AX, BX, CX, and DX). Registers are a type of computer memory used to quickly accept, store, and transfer data and instructions that are being used immediately by the CPU. The general registers AX, BX, CX, and DX are 16-bit. However, they are composed from two smaller registers. For example: AX. The high 8-bit is called AH, and the low 8-bit is called AL.

- ✓ AX - accumulator, and preferred for most operations.
- ✓ BX - base register, typically used to hold the address of a procedure or variable.
- ✓ CX - count register, typically used for looping.
- ✓ DX - data register, typically used for multiplication and division.

In storing string of characters, DX register is used.

mov dx, offset msg1 (msg1 is the name of the variable where you will store the string)
mov ah, 09 (one of the functions in assembly, 09 means displaying the last content saved in dx once it is triggered by the int 21h)
int 21h (interrupt- signal to perform the function 09)
msg1 db "This is a sample string\$" (msg1- the variable name declared above,
 Db – data byte (data is saved in bytes once compiled)
 \$ - all strings shall end in dollar sign to indicate the end of the string)

8. To run your program, in the standard toolbar, click emulate.

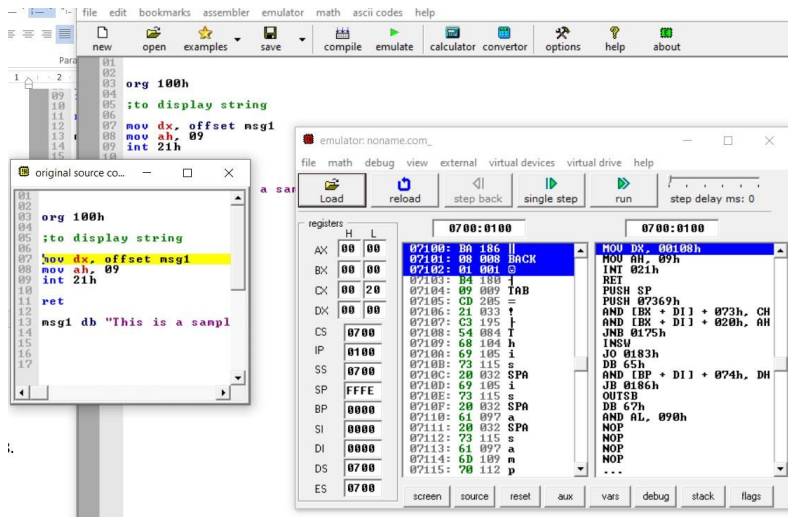


```

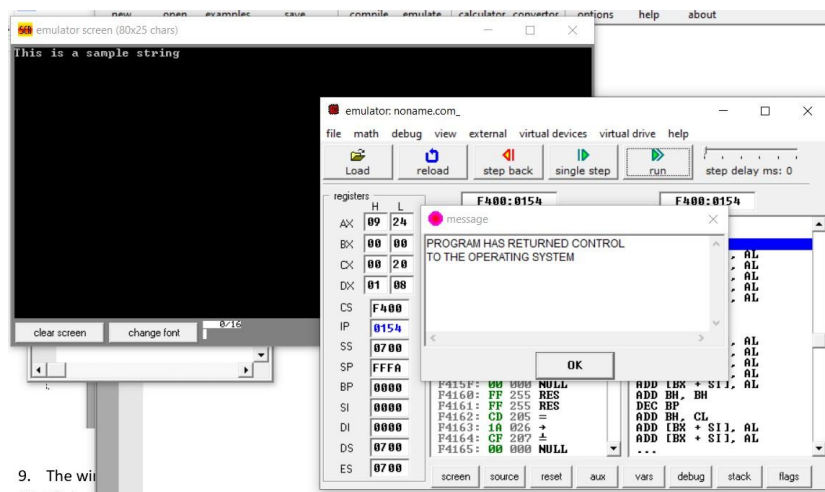
01
02
03 org 100h
04
05 ;to display string
06
07 mov dx, offset msg1
08 mov ah, 09
09 int 21h
10
11 ret
12
13 msg1 db "This is a sample string$"
14
15

```

9. Two windows will appear.



10. The window on the right shows all the registers a CPU has and its current data.
11. Click run.

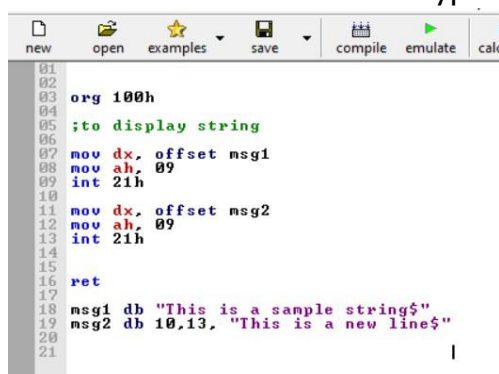


12. The black screen on the left is the Output window. Click OK in the message window which indicates the program has returned control to the Operating System

Note:

If the emulator asks for the registration key, enter the following:
 emu8086 registration key
 User: ISHAAN,glaitm
 Key:27R3VDEFYFX4N0VC3FRTQZX

13. Go back to Code Window then type the following codes:



- 10 - new line
- 13 - cret (carriage return) so the pointer will move to the left.

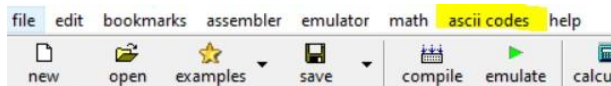
Note: Always use the decimal combination (10, 13) or hexadecimal combination (0Ah, 0Dh) in displaying string in new line.

```

01
02
03 org 100h
04
05 ;to display string
06
07 mov dx, offset msg1
08 mov ah, 09
09 int 21h
10
11 mov dx, offset msg2
12 mov ah, 09
13 int 21h
14
15
16 ret
17
18 msg1 db "This is a sample string$"
19 msg2 db 0AH,0DH, "This is a new line$"
20

```

- 14. Click ascii codes in the standard toolbar to know the decimal and hexadecimal values of the characters.



ascii codes									
000: null	032: spa	064: @	096: `	128: C	160: á	192: Ì	224: α		
001: �	033: !	065: A	097: a	129: �	161: â	193: Í	225: β		
002: �	034: "	066: B	098: b	130: �	162: �	194: Î	226: γ		
003: �	035: #	067: C	099: c	131: �	163: �	195: Ï	227: δ		
004: �	036: \$	068: D	100: d	132: �	164: �	196: �	228: Σ		
005: �	037: %	069: E	101: e	133: �	165: �	197: �	229: σ		
006: �	038: &	070: F	102: f	134: �	166: �	198: �	230: μ		
007: beep	039: '	071: G	103: g	135: �	167: �	199: �	231: τ		
008: back	040: <	072: H	104: h	136: �	168: �	200: �	232: θ		
009: tab	041: >	073: I	105: i	137: �	169: �	201: �	233: �		
010: newl	042: *	074: J	106: j	138: �	170: �	202: �	234: Ω		
011: �	043: +	075: K	107: k	139: �	171: �	203: �	235: �		
012: �	044: ,	076: L	108: l	140: �	172: �	204: �	236: �		
013: cret	045: -	077: M	109: m	141: �	173: �	205: �	237: �		
014: �	046: .	078: N	110: n	142: �	174: �	206: �	238: �		
015: �	047: /	079: O	111: o	143: �	175: �	207: �	239: �		
016: �	048: 0	080: P	112: p	144: �	176: �	208: �	240: �		
017: �	049: 1	081: Q	113: q	145: �	177: �	209: �	241: �		
018: �	050: 2	082: R	114: r	146: �	178: �	210: �	242: �		
019: !!	051: 3	083: S	115: s	147: �	179: �	211: �	243: �		
020: �	052: 4	084: T	116: t	148: �	180: �	212: �	244: �		
021: �	053: 5	085: U	117: u	149: �	181: �	213: �	245: �		
022: �	054: 6	086: V	118: v	150: �	182: �	214: �	246: �		
023: �	055: 7	087: W	119: w	151: �	183: �	215: �	247: �		
024: �	056: 8	088: X	120: x	152: �	184: �	216: �	248: �		
025: �	057: 9	089: Y	121: y	153: �	185: �	217: �	249: �		
026: �	058: :	090: Z	122: z	154: �	186: �	218: �	250: �		
027: �	059: ;	091: [	123: [	155: �	187: �	219: �	251: �		
028: �	060: <	092: \	124: \	156: �	188: �	220: �	252: �		
029: ++	061: =	093:]	125:]	157: �	189: �	221: �	253: �		
030: �	062: >	094: ^	126: ^	158: �	190: �	222: �	254: �		
031: �	063: ?	095: _	127: _	159: �	191: �	223: �	255: res		

00: null	20: spa	40: @	60: `	80: Ç	A0: Á	C0: ª	E0: α
01: �	21: !	41: A	61: a	81: Û	A1: Í	C1: º	E1: ß
02: �	22: "	42: B	62: b	82: Ü	A2: Î	C2: º	E2: �
03: �	23: #	43: C	63: c	83: Ý	A3: Ï	C3: º	E3: �
04: �	24: \$	44: D	64: d	84: à	A4: Ñ	C4: º	E4: Σ
05: �	25: %	45: E	65: e	85: á	A5: Ñ	C5: º	E5: σ
06: �	26: &	46: F	66: f	86: â	A6: Ñ	C6: º	E6: μ
07: beep	27: '	47: G	67: g	87: ç	A7: º	C7: º	E7: τ
08: back	28: (	48: H	68: h	88: è	A8: º	C8: º	E8: �
09: tab	29:)	49: I	69: i	89: é	A9: º	C9: º	E9: �
0A: newl	2A: *	4A: J	6A: j	8A: ê	AA: º	CA: º	EA: Ω
0B: �	2B: +	4B: K	6B: k	8B: ë	AB: º	CB: º	EB: δ
0C: �	2C: ,	4C: L	6C: l	8C: ì	AC: º	CC: º	EC: œ
0D: �	2D: -	4D: M	6D: m	8D: í	AD: º	CD: º	ED: ø
0E: �	2E: .	4E: N	6E: n	8E: î	AE: º	CE: º	EE: €
0F: �	2F: /	4F: O	6F: o	8F: ï	AF: º	CF: º	EF: �
10: �	30: 0	50: P	70: p	90: É	B0: º	D0: º	F0: �
11: �	31: 1	51: Q	71: q	91: æ	B1: º	D1: º	F1: ±
12: �	32: 2	52: R	72: r	92: Å	B2: º	D2: º	F2: �
13: �	33: 3	53: S	73: s	93: Ö	B3: º	D3: º	F3: �
14: �	34: 4	54: T	74: t	94: Ø	B4: º	D4: º	F4: ∫
15: �	35: 5	55: U	75: u	95: ò	B5: º	D5: º	F5: �
16: �	36: 6	56: V	76: v	96: ô	B6: º	D6: º	F6: �
17: �	37: 7	57: W	77: w	97: ù	B7: º	D7: º	F7: �
18: �	38: 8	58: X	78: x	98: ý	B8: º	D8: º	F8: �
19: �	39: 9	59: Y	79: y	99: ÿ	B9: º	D9: º	F9: �
1A: �	3A: :	5A: Z	7A: z	9A: ü	BA: º	DA: º	FA: �
1B: �	3B: ;	5B: [	7B: <	9B: é	BB: º	DB: º	FB: �
1C: �	3C: <	5C: \	7C: i	9C: ê	BC: º	DC: º	FC: �
1D: �	3D: =	5D:]	7D: >	9D: ë	BD: º	DD: º	FD: �
1E: �	3E: >	5E: ^	7E: ~	9E: ð	BE: º	DE: º	FE: �
1F: �	3F: ?	5F: _	7F: ª	9F: �	BF: º	DF: º	FF: res

LABORATORY ACTIVITY:

Program 1 (5pts) – Student 1 as sample data

Display your student information.

Name: JUAN DELA CRUZ

Age: 17

Course: BSIT

Section: BSIT-A

Subject: PLATFORM TECHNOLOGIES

Address: BAGONG SILANG CALOOCAN CITY

Code:

```
org 100h
```

```
;to display string
```

```
mov dx, offset msg1
mov ah, 09
int 21h
```

```
mov dx, offset msg2
mov ah, 09
int 21h
```

```
mov dx, offset msg3
mov ah, 09
int 21h
```

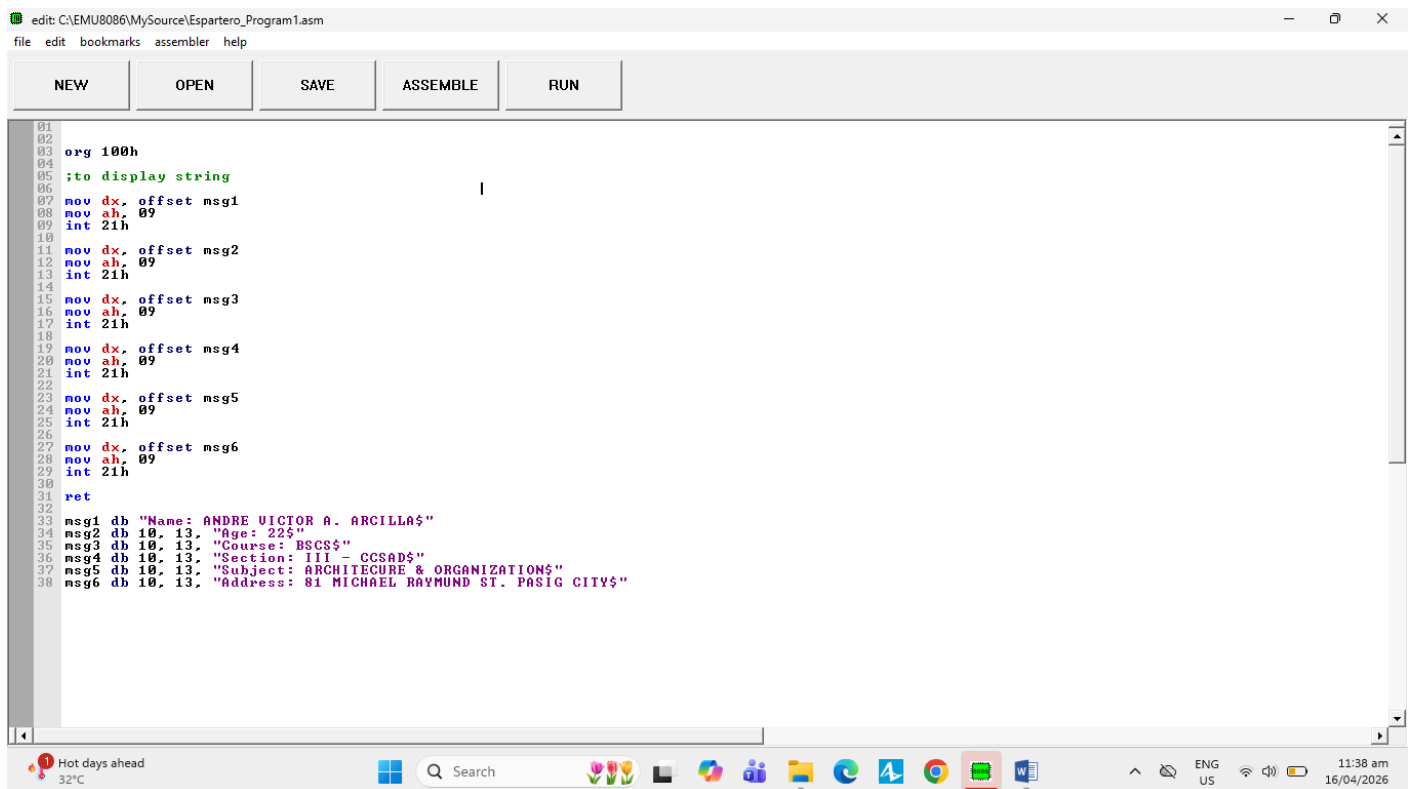
```
mov dx, offset msg4
mov ah, 09
int 21h
```

```
mov dx, offset msg5
mov ah, 09
int 21h
```

```
mov dx, offset msg6
mov ah, 09
int 21h
```

```
ret
```

```
msg1 db "Name: ANDRE VICTOR A.
ARCILLA$"
msg2 db 10, 13, "Age: 22$"
msg3 db 10, 13, "Course: BSCS$"
msg4 db 10, 13, "Section: III - CCSAD$"
msg5 db 10, 13, "Subject: ARCHITECURE &
ORGANIZATION$"
msg6 db 10, 13, "Address: 81 MICHAEL
RAYMUND ST. PASIG CITY$"
```



edit: C:\EMU8086\MySource\Espartero_Program1.asm

file edit bookmarks assembler help

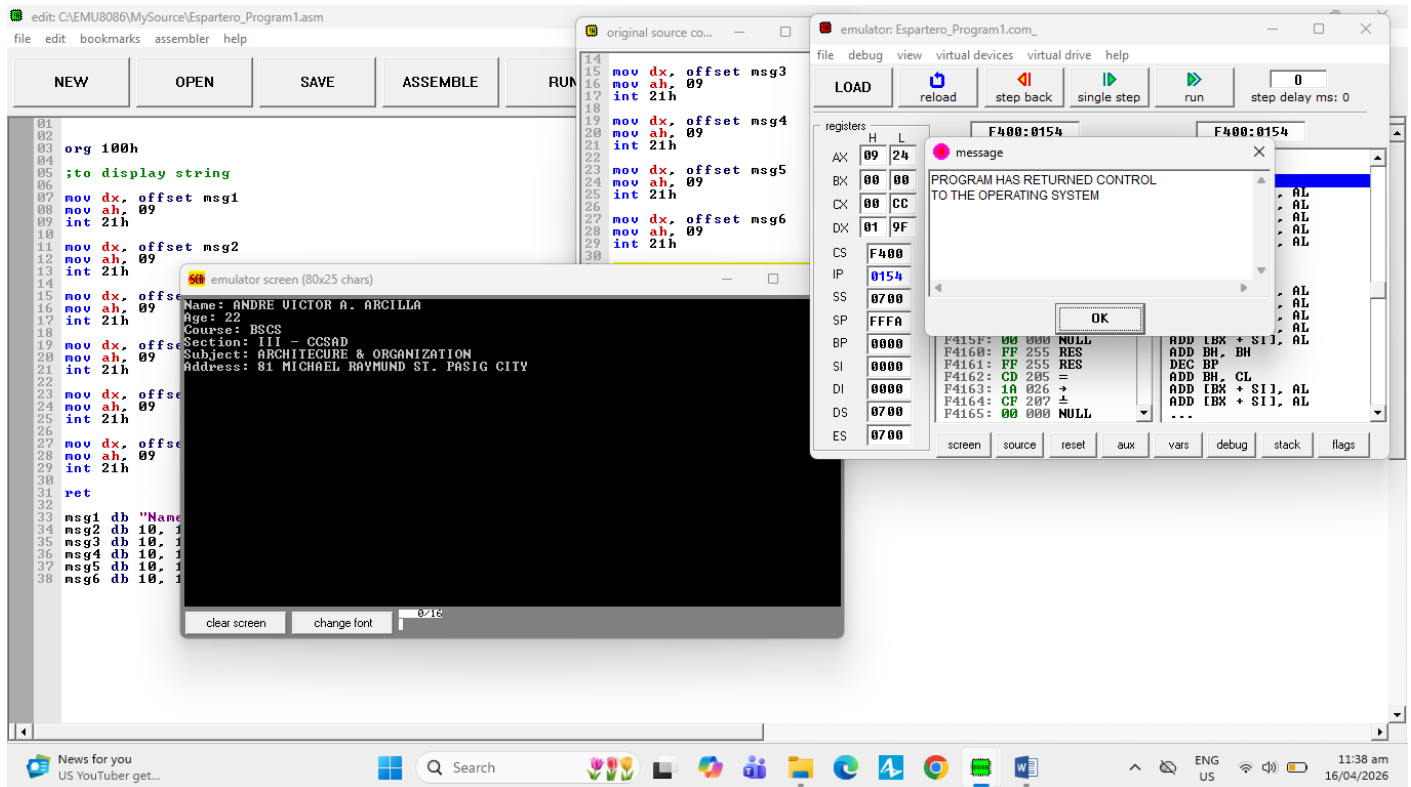
NEW OPEN SAVE ASSEMBLE RUN

```
01
02 org 100h
03
04 ;to display string
05
06
07 mov dx, offset msg1
08 mov ah, 09
09 int 21h
10
11 mov dx, offset msg2
12 mov ah, 09
13 int 21h
14
15 mov dx, offset msg3
16 mov ah, 09
17 int 21h
18
19 mov dx, offset msg4
20 mov ah, 09
21 int 21h
22
23 mov dx, offset msg5
24 mov ah, 09
25 int 21h
26
27 mov dx, offset msg6
28 mov ah, 09
29 int 21h
30
31 ret
32
33 msg1 db "Name: ANDRE VICTOR A. ARCILLA$"
34 msg2 db 10, 13, "Age: 22$"
35 msg3 db 10, 13, "Course: BSCS$"
36 msg4 db 10, 13, "Section: III - CCSAD$"
37 msg5 db 10, 13, "Subject: ARCHITECURE & ORGANIZATION$"
38 msg6 db 10, 13, "Address: 81 MICHAEL RAYMUND ST. PASIG CITY$"
```

Hot days ahead 32°C

Search

ENG US 11:38 am 16/04/2026



Program 2 (10 pts) – Student 2 as example data

Using decimal or hexadecimal values in ASCII codes, display the following:

full name (FIRST NAME MIDDLE NAME LAST NAME)

birthday (MONTH DATE, YEAR)

Life Quotation/ Bible verse

Sample output:

JOMARISS BACCAY PLAN

August 10, 1989

Jeremiah 29:11 "For I know the plans I have for you" declares the Lord, "plans to prosper you not to harm you, plans to give you hope and a future."

CODE:

.MODEL SMALL

.STACK 100H

.DATA

; "RALPH ISRA PABILONA ESPARTERO"

; R:52, A:41, L:4C, P:50, H:48, sp:20, I:49, S:53, R:52, A:41, sp:20

; P:50, A:41, B:42, I:49, L:4C, O:4F, N:4E, A:41, sp:20

; E:45, S:53, P:50, A:41, R:52, T:54, E:45, R:52, O:4F, CR:0D, LF:0A, \$:24

NAME_STR DB 52H, 41H, 4CH, 50H, 48H, 20H, 49H, 53H, 52H, 41H, 20H

DB 50H, 41H, 42H, 49H, 4CH, 4FH, 4EH, 41H, 20H

DB 45H, 53H, 50H, 41H, 52H, 54H, 45H, 52H, 4FH, 0DH, 0AH, 24H

; "September 25, 2005"

; S:53, e:65, p:70, t:74, e:65, m:6D, b:62, e:65, r:72, sp:20

; 2:32, 5:35, :2C, sp:20, 2:32, 0:30, 0:30, 5:35, CR:0D, LF:0A, \$:24

BDAY_STR DB 53H, 65H, 70H, 74H, 65H, 6DH, 62H, 65H, 72H, 20H, 32H, 35H, 2CH, 20H,

32H, 30H, 30H, 35H, 0DH, 0AH, 24H

; Quotation

QUOTE_STR DB 50H, 68H, 69H, 6CH, 69H, 70H, 70H, 69H, 61H, 6EH, 73H, 20H, 34H, 3AH,
31H, 33H, 20H, 22H, 49H, 20H

DB 63H, 61H, 6EH, 20H, 64H, 6FH, 20H, 61H, 6CH, 6CH, 20H, 74H, 68H, 69H, 6EH,
67H, 73H, 20H, 74H, 68H

DB 72H, 6FH, 75H, 67H, 68H, 20H, 43H, 68H, 72H, 69H, 73H, 74H, 20H, 77H, 68H,
6FH, 20H, 73H, 74H, 72H

DB 65H, 6EH, 67H, 74H, 68H, 65H, 6EH, 73H, 20H, 6DH, 65H, 22H, 0DH, 0AH, 24H

.CODE

MAIN PROC

; Initialize Data Segment

MOV AX, @DATA

MOV DS, AX

; Display Full Name

LEA DX, NAME_STR

MOV AH, 09H

INT 21H

; Display Birthday

LEA DX, BDAY_STR

MOV AH, 09H

INT 21H

; Display Quotation

LEA DX, QUOTE_STR

MOV AH, 09H

INT 21H

; Terminate program

MOV AH, 4CH

INT 21H

MAIN ENDP

END MAIN

